`publish_pipeline`

Scenario 1: a provider onboards a PDF. Nobody has asked anything yet.

```
step 1   ingest the PDF, extract passages ONCE
step 2   in parallel:
           2a  passages ──▶ Beckn catalogue metadata   (small; goes to the network)
           2b  passages ──▶ embeddings ──▶ vector DB   (large; stays with the provider)
step 3   /catalog/publish ──▶ the network layer
```

> **The manager document is** `STEP2_WALKTHROUGH.md` — a step-by-step runthrough of 2a and 2b with real values from a real run, and the full vector-DB technicality. Read that one. This file is orientation and how to run it.

In plain language first

Onboarding a bulletin is like cataloguing a library. A new bulletin arrives, and you do two unrelated things to it at the same time:

1. You write an **index card** — which crops it covers, which districts, which languages, how often it is refreshed. Small. That card goes out to the network so anyone can find you.
2. You put the **bulletin itself on a shelf** in your own building, arranged so you can find any paragraph inside it later. Nobody outside gets the bulletin.

Nobody has asked you anything yet. You do both because the bulletin was published, and you do it the same way regardless of who eventually asks or what they ask about.

The reason there are two branches at all — rather than one big index — is that the index card **cannot contain the bulletin**. A catalogue is cached by every consumer that fetched it, so advisory text placed inside it goes stale silently and there is no mechanism to recall it. Advice therefore has to be fetched live from the provider, which is exactly what branch 2b exists to serve.

Run it

From a clean machine, start to finish:

```
git clone https://github.com/td1807/publish_pipeline.git
cd publish_pipeline

python3.11 -m venv .venv                      # any Python >= 3.10
.venv/bin/pip install -r requirements.txt     # ~3 GB: torch, then the e5 model on first run

.venv/bin/python main.py --all --fresh
```

Then the tests:

```
.venv/bin/pytest tests/test_v4.py -q -m "not semantic"   # 43 tests, ~15s
.venv/bin/pytest tests/test_v4.py -q -m semantic         # 1 test, needs the model
```

Three things about that first block are worth knowing, because each one produces a confusing error rather than an obvious one:

- **Python 3.10 is the floor, and macOS ships 3.9.** There is no `python` or `pip` on a stock macOS PATH at all, only `python3`, and that one is 3.9 — which is why `python …` and `pip …` fail with `command not found` before reaching any code here, and why `pip3 install` reports "no matching distribution" for packages that plainly

exist. Calling the venv's own interpreter by path, as above, sidesteps all of it. `main.py` checks the version itself and says so plainly if it is too old.

- **Run `main.py`, not `-m publish_pipeline.run_scenario1`.** The module form still works, but only from the *parent* of the checkout, since the modules import each other relatively. `main.py` puts the parent on `sys.path` for you so the command works from inside the checkout. Either way, the directory must stay named `publish_pipeline`.
- **The first run is slow and needs the network.** Roughly 700 MB of torch at install time and ~2.2 GB of `intfloat/multilingual-e5-large` on first use. Both are cached afterwards (the model in `~/.cache/huggingface`), so later runs start immediately. To see the whole pipeline without either download, use `EMBEDDING_BACKEND=lexical`, which skips the model entirely and stamps `semantic=False` on every result so a degraded run cannot be mistaken for a real one.

Nothing else has to be running: Qdrant is embedded in the process against a local `.qdrant/` directory, there is no server, no Docker, and no port. That directory is not in git — `--fresh` rebuilds it. It is also locked while a run is in progress, so run one at a time; if a run is killed hard, delete `.qdrant/` and re-run with `--fresh`. Offline, prefix commands with `HF_HUB_OFFLINE=1` so sentence-transformers uses the cache instead of checking for a newer revision.

Saved output from a real run is checked in, so the walkthrough can be read without running anything:

- `evidence/SCENARIO_1_TRANSCRIPT.txt`
- `evidence/publish_payload.json` — the actual `/catalog/publish` body (157 KB as published, 280 KB pretty-printed here)
- **`evidence/resources/` — one JSON file per bulletin**, each carrying that document's resources with their full `resourceAttributes`. Usually the more useful view: "what did *this* bulletin claim?" without scrolling past two other states.

file	state	resources	size
`karnataka.json`	IN-KA	71	168 KB
`up.json`	IN-UP	8	51 KB
`rajasthan.json`	IN-RJ	5	17 KB

- `evidence/message_update.reference.json` — the target shape, kept alongside so a test can diff against it
- `docs_pdf/` — this file and the walkthrough as PDF

To print a single resource's attributes straight to the terminal (matched on any substring of its id — note the order is `…-<category>-<area>`):

```
.venv/bin/python main.py --all --show-resource livestock-in-ka-koppal
```

It runs without the model, and says so

The default is the real `intfloat/multilingual-e5-large` (1024-d, multilingual). `EMBEDDING_BACKEND=lexical` runs with nothing downloaded, but it matches **spellings, not meanings** — no paraphrase, no cross-language — and stamps `semantic=False` on every result it returns. That is deliberate: a plausible-looking similarity score from a bag of character n-grams is the easiest way to mislead a room.

What one real run does

Three real government bulletins, chosen because they differ in ways that show up in the output — `--all --fresh`, on Apple Silicon (`device=mps`):

	Karnataka	Uttar Pradesh	Rajasthan
pages	53	45	30
passages	241	240	29
language mix	`en` 240 / `hi` 1	`en` 100 / **`hi` 140**	`en` 2 / **`hi` 27**
subjects resolved	**83.4%**	56.7%	**24.1%**
districts resolved	31 / 31	75 / 75	32 / 32
passages placed in a district	98.3%	32.1%	55.2%
2a resources	**71**	8	5
capability types	4	4	4
2b vectors	241	240	29
max tokens / passage	254	299	314
2a time	~0.01 s	~0.01 s	~0.02 s
2b time	15–186 s	22–154 s	4–49 s

```
step 3   ACCEPTED ─ 3 catalogues, 84 resources, 161,020-byte payload
         network layer holds resourceAttributes only:
           84 resources · 51 subject URIs · 141 area codes · 13 topics
           · 4 subject categories · 7 weather parameters · 2 languages
         and zero advisory text ─ checked against real sentences from the source
         round-trip verified: every field held exactly as published

totals   510 passages · 84 resources · 510 vectors · 0 refused
```

Why the three states differ so much is the interesting part, and it is real signal rather than a bug:

- **Karnataka publishes 71 resources from 53 pages** because it is organised as per-district advisory sections (`Agromet Advisory for Koppal district`), so almost every passage lands in a district and becomes part of a district-level capability.
- **UP publishes only 8 resources from *more* passages (240)** because it is organised by agro-climatic zone, and its tables list 7–8 districts per row. Those passages are honestly statewide, so they group into a handful of state-level resources — while still naming all 75 districts in `coverageAreas`, so a consumer can narrow down.
- **Rajasthan resolves only 24% of passages to a crop** because it genuinely is mostly weather warnings, not crop advisories. The run prints that as a warning rather than letting a thin catalogue look complete.

And the retrieval works

```
query: "pigeon pea flowers dropping, what should I spray"
  0.892  imd_up_agromet.pdf        p.39   res-…-crop-in-up
         "Pigeon Pea (Flowering) At flowering, scout for Helicoverpa larvae…
          spray Emamectin benzoate 5 SG @ 200 g/ha…"
  0.892  imd_karnataka_agromet.pdf p.16   res-…-crop-in-ka-koppal
  0.851  imd_karnataka_agromet.pdf p.13   res-…-crop-in-ka-kalaburagi
```

The word "tur" never appears in either bulletin — they say "Redgram" and "Pigeon pea". A query using "tur" still finds it (there is a test for exactly that), which is what the 1024-d multilingual model is buying. A Hindi query (धान की फसल में सिंचाई) returns Hindi passages from the UP bulletin.

Files

file	what it is	read it for
`config.py`	every setting, and what each degrades to	"what do I need installed"
`taxonomy/vocab.py`	crop/district/topic vocabulary + alias resolution	**the only thing both branches share**
`taxonomy/ids.py`	resource-id and point-id rules	why a reissue updates instead of duplicating
`taxonomy/data/*.json`	the vocabulary itself	what we can and cannot recognise
`ingest/document_text.py`	file → pages	ingestion, the formats it reads, and when it refuses
`ingest/language.py`	script/language detection + encoding check	the multi-language story
`ingest/passages.py`	pages → `Passage` (text **and** facets in one object)	**why 2a and 2b cannot drift**
`beckn/models.py`	pydantic mirrors of `beckn.yaml`	spec conformance
`beckn/resource_attributes.py`	facets → `resourceAttributes`	**the heart of 2a**
`beckn/catalog.py`	passages → resources → catalogue	capability typing
`beckn/envelope.py`	the `message_update.json` envelope	step 3's payload
`beckn/validate.py`	spec + coherence + no-prose checks	what we refuse to publish
`vectors/embeddings.py`	the model, its prefixes, its 512-token limit	**the vector DB: model, dims**
`vectors/store.py`	Qdrant collection, payload, filters, ids	**the vector DB: schema, filters**
`network_node.py`	stand-in for the network layer	**what it holds**, and the two-hop shape
`publish.py`	step 3	validate-then-deliver
`scenario1.py`	`onboard()`, `publish_all()`, branch timings	the orchestration
`run_scenario1.py`	runnable, narrated	just run it
`tests/test_v4.py`	43 tests	most claims above, as assertions
`tools/md2pdf.py`	optional doc → PDF renderer	regenerating `docs_pdf/`

Nothing here imports from `pipeline.*`, from `pipeline_beckn_v2` or from `publish_pipeline_beckn_v3`. The whole flow reads end to end without following imports into another package.

The design decisions, in one place

1. **One extraction pass feeds both branches.** A `Passage` carries the text (2b's payload) *and* its resolved facets (2a's raw material), and both branches read the same `Passage.facets()` method. Two extractors would drift, and the failure mode is nasty: discovery routes a question to a resource whose stored passages carry a different filter, so the provider searches inside a resource it just advertised and finds nothing. `test_facet_parity` holds the line.
2. **The unit of publication is a capability, not a document.** One resource per *(primary coverage area × capability category)* — "crop advisory for Koppal" — not one per PDF and not one per advisory row. Resource ids derive from *(provider, domain, category, area)* and contain nothing about the bulletin issue, so next Friday's reissue **updates** the same resources instead of minting duplicates.

Because the unit is *(area × category)*, `@type` **is a function of the category, not of the document.** One agromet bulletin therefore publishes four different capability types, and each carries only the attributes its type is for: a crop resource has `agricultureSubjects` and no `weatherParameters`; a forecast resource the reverse. `validate.py` fails a payload where the two disagree.

1. **The catalogue carries metadata only.** No advisory text, not even a snippet. Size is the small reason; staleness is the real one. `validate.py` fails the payload if prose creeps in, and a test greps the real payload for actual sentences from the source bulletins.
2. `resourceAttributes` **is where all the domain work lives — and it is what the network layer holds.** Beckn core leaves it as an open JSON-LD object on purpose (`Attributes`: requires only `@context` and `@type`, `additionalProperties: true`). Everything agriculture-specific goes there: `agricultureSubjects[]` with taxonomy URIs, `coverageAreas[]` as governed codes, `languages[]`, `topics[]`, `weatherParameters[]`, `forecastHorizon`, `updateFrequency`, `geographicGranularity`, plus an `evidence` block.

Stated in the right order: **the network layer holds the full `resourceAttributes`; what it does not hold is document text.**

1. **Areas are governed codes, never invented coordinates.** A polygon nobody surveyed is a polygon we made up, and downstream nothing can tell it from a real boundary. States use real ISO-3166-2 (`IN-KA`). There is a test asserting the payload contains no `coordinates` key at all.
2. **Extraction is rules-first.** The alias table does the work: deterministic, free, auditable, and byte-identical on every re-run — which matters because a churning catalogue republishes for no reason. **No LLM is in the default path, and no code path may mint a subject URI** that the taxonomy did not; `validate.py` checks every `subjectId` against the vocabulary.
3. **Closed-vocabulary fields are validated by membership, not by eyeball.** A topic must *be* a topic from `capabilities.json`. This is strictly stronger than scanning those fields for prose — which matters, because `"Spraying"` is simultaneously a canonical topic name and a word any prose detector would flag.
4. **A document we cannot stand behind is refused, not guessed at.** Three cases reach the same answer: a file no extractor can open, a document with no usable text layer (a scan), and a bulletin from a state the vocabulary does not cover. All three raise `UnusableDocument`, all three print a `REFUSED` line naming the reason, and none of them stops the other documents in the run. The alternative is a resource claiming coverage it cannot serve.

Known limits — read before demoing

- **Parallelism buys nothing measurable, and the run says so.** The two branches genuinely execute concurrently in a thread pool, but 2a costs ~10 ms against 2b's tens of seconds, so wall clock is simply 2b's time — the measured saving is ~0 (ratio 2,325×–27,678× in the latest run). The reason to run them concurrently is architectural (neither branch blocks the other, and either can fail without corrupting the other), **not throughput.** A real speed-up would come from parallelising *within* 2b. `BranchTiming.summary()` prints the ratio rather than claiming a win.
- **2b's timing varies by more than 12× on identical input.** Karnataka measured 14.9 s, 33.5 s, 83.6 s and 186.2 s across four runs on the same laptop (at 155 passages, before the crop-boundary chunking; at 241 passages four runs today gave 16.0, 17.5, 17.9 and 18.4 s) — fastest on an idle machine, slowest with the GPU hot and contended straight after a test run. The table above gives ranges for that reason. Quote no single figure; measure on the target hardware, and expect a dedicated GPU to be both faster and far more consistent.
- **An alias must not be a whole word inside another crop's name.** `gram` was listed as an alias for Bengal gram, and India grows black gram, green gram, horse gram and red gram. Resolution is word-boundary aware, so `tur` inside `turmeric` was never a problem — but `gram` inside `black gram` was, and the Karnataka and UP catalogues advertised Bengal gram coverage from bulletins that never mention chickpea. That is the precise failure `crops.json` warns about at the top of the file: a wrong subject URI routes a farmer to a provider that cannot help, and nothing downstream can tell.

Removing the alias withdraws both false claims. A test now asserts the general rule — every crop name resolves to that crop and no other — with one documented exception, `fodder sorghum`, which also resolves to `sorghum` because it genuinely is sorghum.

- **Passages are cut on crop boundaries as well as on length.** An IMD advisory table runs one crop's advice into the next with no blank line between them, so splitting purely on size produced passages that were about two crops and therefore precisely about neither. `_split_on_crop_change()` starts a new passage at a line naming crops the current one does not share, provided what is already buffered can stand alone; a line naming no crop always continues the run, because table rows rarely repeat their crop.

A crop change can strand a tail too short to survive `MIN_PASSAGE_CHARS`, so a run under that length is folded back into its neighbour. That is not cosmetic: the Karnataka bulletin ends a rain-impact list with "lodging of Banana plant." — 24 characters, and the document's only mention of that crop. Without the fold it was dropped and Banana disappeared from the catalogue. Keeping two crops in one passage is a far smaller cost than losing a line.

Measured across the three bulletins: passages 357 → 510, passages carrying a resolved subject **208 → 344**, passages carrying more than one crop **133 → 68**, and passages placed in a district 243 → 330. Text coverage is unchanged at 99.5% (per bulletin: 99.5% / 99.7% / 97.2%). Nothing was lost from the catalogue: same crops, same topics, same area codes per bulletin, and 74 → 84 resources. The *percentage* placed in a district falls for UP and Rajasthan only because the denominator grew; those absolute counts are 76 → 77 and 16 → 16.

What it did **not** clearly improve is retrieval. Measured over all three bulletins with 14 farmer questions in Hindi and English: **12 unchanged, 1 better, 1 worse** (MRR@5 0.655 → 0.643 when the ground truth demands one specific Hindi passage; see the walkthrough §4.9 for the second, looser scoring and why the absolute figure should not be quoted). Fall armyworm in maize improved from rank 3 to 2; "wilt in bengal gram" fell out of the top 5, into a cluster of black-gram and green-gram passages scoring 0.797–0.807 — a near-tie shuffle, not a structural loss. **The demonstrated gain is in labelling, not ranking.**

An English question does **not** fail against this corpus, though an earlier version of this section claimed it did — that was a measurement error, not a finding. Scored against "did the farmer get correct advice", `okra yellow mosaic virus, what to spray` and `how to protect livestock during heavy rain` both return the right passage at rank 1. What they return is the *English* advice from another state's bulletin rather than the local Hindi passage, because nothing in a bare semantic query says which state the farmer is in. In the real flow that is discovery's job: the consumer picks a provider by area code, and the follow-up search is scoped with `resource_ids`.

- **PDF is what these bulletins arrive as, but not what the code is limited to.** Ingestion goes through MuPDF, which reads **PDF, DOCX, TXT, HTML, EPUB and XPS**; all of those reach the catalogue, and a DOCX bulletin is covered by a test. Nothing downstream of `ingest/` knows what the file was — it consumes pages of text with numbers on them — so format is owned by one module. What does *not* read is the legacy `.doc` binary, spreadsheets and archives; those are refused by name:

```
REFUSED old_bulletin.doc Refusing to publish a catalogue from an unreadable document: old_bulletin.doc could not be opened (...). PDF and DOCX are what these bulletins arrive as; TXT, HTML, EPUB and XPS also read. The legacy .doc binary format, spreadsheets and archives do not — convert to PDF or DOCX and re-ingest.
```

Reading a format and extracting it *well* are different questions. The passage rules key off the IMD bulletin layout — `Major crops | Stage | Pest/disease | Advisories` tables and district headings — not off the file format. A DOCX laid out that way extracts at least as well as the PDF. A spreadsheet of the same data would open and produce nonsense.

- **Three states, and a fourth one is refused rather than guessed.** `_STATE_MARKERS` in `ingest/passages.py` knows Karnataka, Uttar Pradesh and Rajasthan, and `taxonomy/data/districts.json` carries 138 districts across those same three — authored from the three source bulletins. A bulletin from anywhere else cannot be given an area code without inventing a coverage claim, so it is refused by name:

```
REFUSED kerala_prices.pdf Cannot determine which state kerala_prices.pdf covers. Add a marker to _STATE_MARKERS in ingest/passages.py rather than guessing — an area code is a claim about coverage.
```

Adding a state is a marker line plus that state's districts with their aliases and local-script spellings. The marker is a minute; the district vocabulary is the actual work, and it has to be right, because those codes are published as coverage.

- **`context.domain` is not in Beckn v2.** The spec's `Context` has no `domain` property — v2 replaced it with `schemaContext`. We emit it because `message_update.json` carries it. Context is not `additionalProperties: false`, so it is tolerated, but it is a house extension rather than spec.
- **`CatalogPublishAction` is marked `deprecated: true`** in `core-v2.0.0-lts`, even though `/catalog/publish` remains the documented publish path and the spec offers no replacement.
- **Two documents cited throughout are not in this repository.** `beckn.yaml` is the Beckn core spec (`core-v2.0.0-lts`), which the models in `beckn/models.py` mirror by hand rather than vendor — so conformance here is checked against those models, not machine-checked against the published schema. `message_update.json` is the target payload this work was given to match; it is checked in as `evidence/message_update.reference.json`, and a test diffs the built envelope against it.
- **The schema URLs do not resolve.** `https://schemas.openagrinet.global/...` fails DNS today, and `OpenAgriNet/network-specs` contains only a 15-byte README. `@context` is required by the spec, so we emit the intended URL rather than substitute one that happens to resolve.
- **`provider.availableAt` is omitted.** Beckn's `Location` requires a `geo` geometry and we have no surveyed boundary for any state. Coverage is expressed as area codes inside `resourceAttributes` instead. The supplied `message_update.json` has a hand-drawn Karnataka bounding box; we do not reproduce that.
- **District codes are not LGD.** LGD is the correct national scheme and its codes are numeric; we do not have the master loaded, so districts are emitted under `OPENAGRI-DISTRICT` with a readable code we can vouch for. Swapping in LGD is a two-field change in `taxonomy/vocab.py`.
- **Devanagari in these files has an encoding defect, and the two files have different ones.** The fonts are proper Unicode (Nirmala UI, Mangal), but the producing tool mis-maps glyphs. The UP bulletin emits `प्रदि` where `प्रदेश` belongs. The Rajasthan bulletin has a worse and more systematic defect: `ब` and `ि` are swapped, and the i-matra is emitted at the position it is *drawn* — left of its consonant — rather than in logical order, so `सिरोही` arrives as `बसरोही` and `बैंगन` as `िैंगन`. Because the words it corrupts are crop and district names, it cost real coverage: the catalogue advertised 4 crops for a bulletin that discusses 8.

`repair_devanagari()` in `ingest/language.py` undoes it, and `repair_encoding()` in `scenario1.py` decides whether to trust the result — it scores both versions against the crop and district vocabulary and keeps the repair only if more known terms match. Measured: **Rajasthan 28 → 37 terms, applied; UP 121 → 78, refused.** The same transform run over the UP bulletin would destroy it, which is exactly why the gate exists rather than a blanket "fix Devanagari" pass. A repair that fires says so on every run:

```
encoding fix applied — 9 more vocabulary term(s) now match (28 → 37)
```

Measured defect rate before repair: **0.7% of Devanagari words in UP, 0.1% in Rajasthan**. Embedding is robust to it; exact-term lookup on an affected word is not. The alias tables absorb the specific corrupted spellings that appear, and `check_devanagari_encoding()` reports density on every run so a genuinely legacy-font file would be caught.

- **Hindi vs Marathi is not distinguished.** Devanagari is shared between them (plus Nepali, Sanskrit, Konkani). We report `hi` and set `ambiguous=True` with the sibling list, rather than shipping a model to guess.
- **Embedded Qdrant ignores payload indexes** and evaluates filters by scanning. Results are correct; latency is not a production figure. `VectorIndex.describe()` says which mode it is in for exactly this reason.
- **No Beckn signatures.** `AuthorizationHeader`, `AckSignatureHeader` and `context.key` are not implemented. Discovery trusts whoever calls it, and scoping a vector search by `resource_ids` **does not authenticate** that the caller was ever given them — resource ids are public. This is the largest gap and whoever builds the production consumer leg must know it.
- **`network_node.py` is a model, not an implementation.** No registry lookup, no signature verification, no multi-provider fan-out; persistence is a dict.

- **Scenario 2 is not built.** The follow-up loop (step 5 of the pipeline diagram) is out of scope here — scenario 1 involves no question. What is proven is that branch 2b's index *can* answer one: `run_scenario1` ends with a retrieval smoke check, and `test_retrieval_can_be_scoped_to_advertised_resources` demonstrates the discovery→filter→answer path.